Foundational concepts of machine learning

Lecture Notes

Giorgio Gambosi

Master's Degree in Computer Science

University of Rome "Tor Vergata"

A.Y. 2026–2027

Contents

1 Introductory concepts

Machine learning rests on a simple observation: data is rarely random. Whether you analyse images, text, sensor measurements, or human behaviour, the data you collect contains patterns reflecting the underlying process that generated it. The goal of machine learning is to infer such patterns from data without explicitly programming the rules.

From a collection of observations we construct models: mathematical abstractions capturing the relationships present in the data, used for prediction and decision-making on new, unseen data.

1.1 Three paradigms of machine learning

1.1.1 Supervised Learning

In supervised learning we work with paired data: observations (as feature sets) and corresponding target values. The goal is to learn a mapping from inputs to targets and predict unseen cases. For instance, given houses described by square footage, number of bedrooms, and location, each with a known selling price, we learn a function mapping features to prices and use it to estimate the price of a new house.

Supervised learning splits into two categories depending on the nature of the target:

Regression The target is a real number (continuous): prices, temperatures, stock values, or other continuous quantities.

Classification The target is a discrete category: spam detection, handwritten digit recognition, diagnosis, sentiment analysis.

In both cases, we define a model: either a function $y(\mathbf{x})$ outputting a prediction directly, or a probability distribution $p(t|\mathbf{x})$ quantifying uncertainty about the target value. Training data are then used to adjust the model's parameters.

1.1.2 Unsupervised Learning

Unsupervised learning works without explicit targets, looking for structure in unlabelled data: clustering groups similar items; density estimation models the distribution generating the data; dimensionality reduction projects data onto lower-dimensional representations preserving key information (**Principal Component Analysis (PCA)**, for instance, retains the top k orthogonal directions of greatest variance).

1.1.3 Reinforcement Learning

In reinforcement learning, rather than working with a fixed dataset, an agent learns through interaction with an environment, repeatedly choosing actions and adapting its behaviour to maximize cumulative reward. At each step the agent observes the current state and selects an action; the environment returns a reward and a new state. These signals guide the agent toward an optimal policy $\pi : S \to A$ maximizing expected cumulative reward:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{1}$$

where $\gamma \in [0, 1]$ is a discount factor balancing immediate versus future rewards, and r_t is the reward at step t. The central tension of this framework is the exploration-exploitation tradeoff, and since it differs substantially from the other two paradigms these notes set it aside.

1.2 Data representation

Every machine learning method starts with a data representation choice. Real-world entities (images, documents, customers, transactions, biological samples) must be converted into feature vectors. A feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ is a collection of d numerical measurements describing an object, and its choice affects both prediction quality and computational efficiency. In image classification, for instance, raw pixel intensities yield millions of noisy features, while edge orientations or textures may be more informative. This is why feature engineering (combining, selecting or learning features) is often as important as the choice of learning algorithm.

1.3 The learning process and generalization

Across all paradigms we start with a training set $\mathcal{T} = (\mathbf{x}, \mathbf{t})$ containing n examples. In supervised learning, each example pairs an input feature vector $\mathbf{x}_i$ with its target value t_i; in unsupervised learning, we have only the feature vectors $\mathbf{x} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$.

The key challenge is not fitting the training data but generalizing to unseen data. A model memorising the training data fails on new examples; an overly simple one misses important patterns. The capacity of a model determines its flexibility, the bias-variance tradeoff reflects the tension between underfitting and overfitting, and regularisation penalises complexity to improve generalization. These notions are made precise in the rest of these notes.

1.4 Formal definition of a supervised learning task

A supervised learning task involves two key sets: the domain set $\mathcal{X}$ and the label set $\mathcal{Y}$. The domain set collects the objects to be labelled; each object is modelled as an array of features $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, where d is the dimensionality (more complex structures, such as sequences or graphs, are also possible). The label set contains all possible output values: if $\mathcal{Y}$ is continuous we face a regression task, and when $\mathcal{Y}$ is discrete a classification task — binary when $|\mathcal{Y}| = 2$ and multi-class when $|\mathcal{Y}| > 2$.

The learning algorithm $\mathcal{A}$ receives a training set $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$, organised into a feature matrix $\mathbf{X}$ (objects as rows) and a target vector $\mathbf{t}$:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_n \end{pmatrix}. \tag{2}$$

The aim is to predict labels for unseen elements $\overline{\mathbf{x}} \in \mathcal{X}$, either by directly using the training set at prediction time or, more generally, by deriving from it a predictor used for the prediction.

1.5 Relations between items and their targets

Both the objects we observe and their labels come from some underlying stochastic process. The input distribution p_M describes the sampling likelihood of objects, while the simplest assumption for the target is a deterministic ground truth function $f : \mathcal{X} \to \mathcal{Y}$ such that $t = f(\mathbf{x})$ exactly. More realistically, target values may depend on unobserved variables, or observations may be noisy: the same object may then legitimately receive different labels. This is captured by a conditional label distribution $p_C(\cdot|\mathbf{x})$, which, together with p_M, induces the joint distribution $p_J(\mathbf{x}, t) = p_M(\mathbf{x})\, p_C(t|\mathbf{x})$ over object-label pairs. The functional setting is the special case in which $p_C(\cdot|\mathbf{x})$ places all its mass on $f(\mathbf{x})$.

1.6 Functional predictors

In the most natural definition, a **functional predictor** is a map $h : \mathcal{X} \to \mathcal{Y}$ assigning to each input $\mathbf{x}$ a prediction $\hat{y}$:

$$\mathbf{x} \xrightarrow{h} \hat{y} \in \mathcal{Y}. \tag{3}$$

For reasons that will become clear, however, it is in most cases defined as a map $h : \mathcal{X} \to \mathbb{R}$ assigning a real-valued **score** $y = h(\mathbf{x})$ to every input, with the final prediction $\hat{y} \in \mathcal{Y}$ obtained through a task-dependent **decision rule** $d : \mathbb{R} \to \mathcal{Y}$:

$$\mathbf{x} \xrightarrow{h} y \xrightarrow{d} \hat{y} \in \mathcal{Y}. \tag{4}$$

There are three different strategies for constructing a prediction function.

1.6.1 Direct prediction computation

In the simplest approach, a predefined algorithm computes $y = h(\mathbf{x}, \mathcal{T})$ at prediction time using both the new item and the entire training set. No explicit learning phase exists: training examples are stored and referenced during prediction.

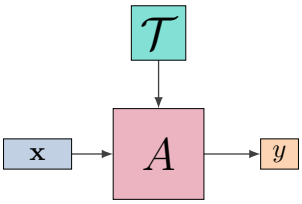


Figure 1: Direct prediction: the algorithm A takes both the new input $\mathbf{x}$ and the entire training set $\mathcal{T}$ at prediction time, producing output y without a separate learning phase.

A classic example is the **k-nearest neighbours** (k-NN) algorithm: to classify a new item $\mathbf{x}$, find the k training objects closest to $\mathbf{x}$ under some distance measure and output their majority class. The approach is simple, locally adaptive, and model-free.

1.6.2 Model-based learning approach

The second, more typical approach defines a **hypothesis class** $\mathcal{H}$, a predefined family of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, and uses a learning algorithm $\mathcal{A}$ to select the function $h_{\mathcal{T}} \in \mathcal{H}$ best approximating the training examples. Prediction then simply applies $h_{\mathcal{T}}(\mathbf{x})$.

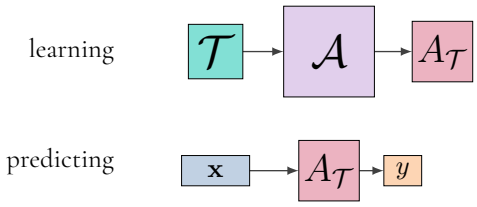


Figure 2: Model-based learning. During training (top), algorithm $\mathcal{A}$ processes $\mathcal{T}$ and produces a trained predictor $A_{\mathcal{T}}$. During prediction (bottom), $A_{\mathcal{T}}$ maps a new input $\mathbf{x}$ to an output y efficiently, without revisiting the training data.

Often $\mathcal{H}$ consists of **parametric functions**, sharing the same structure but differing in parameter values: we write $\mathcal{H}_{\boldsymbol{\theta}}$ with $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_m)$, and learning reduces to finding the parameter values best fitting the training data. In linear regression, for instance, the target is a weighted sum of features:

$$y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^T \overline{\mathbf{x}} \tag{5}$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_d)^T$ is the parameter vector and $\overline{\mathbf{x}} = (1, x_1, \ldots, x_d)^T$ includes a constant for the bias term; learning finds the weights minimizing prediction error on the training set.

1.6.3 Ensemble learning approach

The third approach constructs multiple predictors and combines their predictions; this **ensemble** strategy often outperforms any individual predictor, because different models capture complementary information. First, s predictors $A_{\mathcal{T}}^{(1)}, \ldots, A_{\mathcal{T}}^{(s)}$ are derived from the training set; second, each is assigned a weight $w^{(i)}$ reflecting its estimated quality; third, at prediction time their outputs $y^{(i)} = h_{\mathcal{T}}^{(i)}(\mathbf{x})$ are combined using the weights.

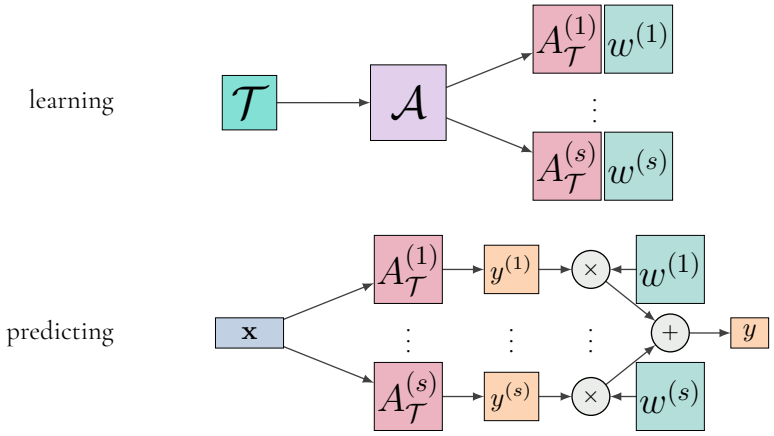


Figure 3: Ensemble learning. Training (top) produces s independent predictors $A_{\mathcal{T}}^{(1)}, \ldots, A_{\mathcal{T}}^{(s)}$ with associated weights $w^{(1)}, \ldots, w^{(s)}$. At prediction time (bottom), each predictor votes and the weighted combination yields the final output y.

For regression, we compute a weighted average,

$$h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)} h^{(i)}(\mathbf{x}) \tag{6}$$

where the weights satisfy $\sum_{i=1}^{s} w^{(i)} = 1$ and $w^{(i)} \geq 0$. For classification, we use **weighted voting**, selecting the class receiving the most weight:

$$h(\mathbf{x}) = \operatorname*{argmax}_{y \in \mathcal{Y}} \sum_{i=1}^{s} w^{(i)} \mathbb{1}[h^{(i)}(\mathbf{x}) = y]. \tag{7}$$

Direct methods are flexible and locally adaptive, model-based methods efficient and interpretable, ensemble methods robust and often more accurate.

1.7 Probabilistic predictors

A **probabilistic predictor** is a map

$$h : \mathcal{X} \to \mathcal{D}(\mathcal{Y}), \tag{8}$$

where $\mathcal{D}(\mathcal{Y})$ denotes a set of probability distributions over $\mathcal{Y}$: for each input $\mathbf{x}$, the predictor returns a distribution $h(\mathbf{x}) = p_P(\cdot|\mathbf{x})$ on $\mathcal{Y}$, rather than a single score. A functional predictor is the special case in which all the probability mass sits on a single value. A hard prediction is obtained by applying a **decision rule** $d : \mathcal{D}(\mathcal{Y}) \to \mathcal{Y}$ directly to the distribution p_P, exactly parallel to the functional case:

$$\mathbf{x} \xrightarrow{h} p_P \xrightarrow{d} \hat{y} \in \mathcal{Y}. \tag{9}$$

Regression. $\mathcal{Y} = \mathbb{R}$, and the natural decision rule is the **posterior mean**

$$\hat{y} = d(p_P) = \mathop{\mathbb{E}}_{t \sim p_P(\cdot|\mathbf{x})}[t]\,. \tag{10}$$

Binary classification. $\mathcal{Y} = \{-1, 1\}$, and the natural decision rule is the posterior mode,

$$\hat{y} = d(p_P) = \operatorname*{argmax}_{t \in \mathcal{Y}} p_P(t|\mathbf{x}), \tag{11}$$

Multiclass classification. $\mathcal{Y} = \{y_1, \ldots, y_K\}$, and the natural decision rule is again the posterior mode,

$$\hat{y} = d(p_P) = \operatorname*{argmax}_{t \in \mathcal{Y}} p_P(t|\mathbf{x}). \tag{12}$$

The distribution p_P can be returned explicitly (the values $p_P(y_k)$ for a discrete $\mathcal{Y}$), through the parameters of a parametric family (e.g. mean and variance of a Gaussian), or as a method computing $p_P(t)$ for any t. The three prediction strategies listed above exist also in the probabilistic case.

1.7.1 Direct prediction.

An algorithm takes $\mathbf{x}$, t, and the entire training set $\mathcal{T}$ at prediction time and returns $p_P(t|\mathbf{x})$ directly, without a separate learning phase.

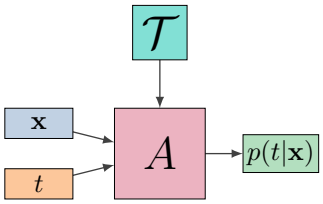


Figure 4: Direct prediction: algorithm A takes the new input $\mathbf{x}$, a target value t, and the entire training set $\mathcal{T}$ at prediction time, producing output $p(t|\mathbf{x})$ without a separate learning phase.

An example is the use of density estimation methods such as Parzen windows in classification, to compute the inverse probabilities $p(\mathbf{x}|y_i)$, from which $p_C(t \mid \mathbf{x})$ follows by Bayes' rule (given a prior distribution $p(t)$).

1.7.2 Model-based learning.

Here the role of the hypothesis class $\mathcal{H}$ is taken by a parametric distribution $\mathcal{D}$, a predefined family of conditional distributions $p(\cdot|\cdot;\boldsymbol{\theta})$. Given a training set $\mathcal{T}$, the learning algorithm $\mathcal{A}$ selects the distribution $p^*(\cdot|\cdot;\mathcal{T})$ best approximating the training examples, by a criterion discussed later. At prediction time, for an item $\mathbf{x}$, the returned distribution is $p_P = p^*(\cdot|\mathbf{x};\mathcal{T}) \in \mathcal{D}$.

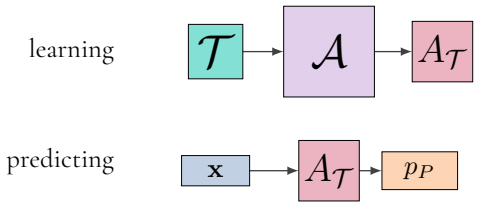


Figure 5: Model-based learning. During training (top), algorithm $\mathcal{A}$ processes $\mathcal{T}$ and produces a trained predictor $A_\mathcal{T}$. During prediction (bottom), $A_\mathcal{T}$ maps a new input $\mathbf{x}$ to a distribution p_P without revisiting the training data.

1.7.3 Ensemble learning.

The fully Bayesian approach, introduced later, works as an ensemble learning approach: rather than selecting a single predictor, you derive $p(\cdot|\mathbf{x})$ by averaging predictions over a family of parametric distributions.

2 Measuring the quality of a predictor: the loss function

Learning means selecting a predictor within some class, which requires measuring how good a prediction is. This is done through the **risk**, the error a prediction incurs on a randomly chosen item, built on a **loss function** $L(h(\mathbf{x}), t)$ measuring how far the prediction for $\mathbf{x}$ is from its true output t. When $h(\mathbf{x})$ is a distribution, the comparison function is called a **scoring rule**, though "loss function" is often used loosely for both.

2.1 Loss function definition, per-element risk, and expected risk

The loss function L compares a prediction y — a score in $\mathbb{R}$, a target value in $\mathcal{Y}$, or a distribution on $\mathcal{Y}$ — with the target value $t \in \mathcal{Y}$, returning the cost of predicting y when the target is t[1]. The **per-element risk** $\mathcal{R} : \mathcal{X} \mapsto \mathbb{R}$ of a predictor h, on a single item $\mathbf{x}$ is the error we expect h to incur when applied on input $\mathbf{x}$: under a functional relation it is simply

$$\mathcal{R}(\mathbf{x}; h) = L(h(\mathbf{x}), f(\mathbf{x})) \tag{13}$$

under a probabilistic relation it is the expectation of $L(h(\mathbf{x}), t)$ over $t \sim p_C(\cdot|\mathbf{x})$,

$$\mathcal{R}(\mathbf{x}; h) = \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \left[L(h(\mathbf{x}), t) \right]. \tag{14}$$

The **expected risk** for the predictor h is the expectation of the per element risk $\mathcal{R}(\mathbf{x})$ over all $\mathbf{x} \sim p_M$

$$\mathcal{R}(h) = \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \left[\mathcal{R}(\mathbf{x}; h) \right] = \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \left[\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \left[L(h(\mathbf{x}), t) \right] \right] = \mathop{\mathbb{E}}_{(\mathbf{x}, t) \sim p_J} \left[L(h(\mathbf{x}), t) \right] \tag{15}$$

this is the average error of the predictor on a randomly chosen item, our main theoretical indicator of model quality. Computing it, however, requires the knowledge of $p_J(\mathbf{x}, t) = p_C(t|\mathbf{x}) p_M(\mathbf{x})$, which is unavailable by assumption; a related indicator, the **empirical risk**, is used in practice.

Four cases arise, depending on the relation between item and target (functional vs. probabilistic) and on the type of predictor (functional vs. probabilistic), governed by a single principle: the **type of predictor** determines the structure of the loss (a value or score y compared with t by a loss function, or a distribution p_P compared with t by a scoring rule), while the **relation model** determines the per-element risk (the loss evaluated at $t = f(\mathbf{x})$, or an expectation over $p_C(\cdot|\mathbf{x})$). Table 1 summarizes the four cases.

Setting	Loss	Per-element risk $\mathcal{R}(\mathbf{x}; h)$	Expected risk $\mathcal{R}(h)$
FF case functional predictor functional relation	$L(y,\, t)$	$L\big(h(\mathbf{x}),\, f(\mathbf{x})\big)$	$\mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \big[L\big(h(\mathbf{x}), f(\mathbf{x})\big) \big]$
FP case functional predictor probabilistic relation	$L(y,\, t)$	$\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \big[L\big(h(\mathbf{x}), t\big) \big]$	$\mathop{\mathbb{E}}_{(\mathbf{x}, t) \sim p_J} \big[L\big(h(\mathbf{x}), t\big) \big]$
PF case probabilistic predictor functional relation	$S\big(p_P, t\big)$	$S\big(h(\mathbf{x}), f(\mathbf{x})\big)$	$\mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} \big[S\big(h(\mathbf{x}), f(\mathbf{x})\big) \big]$
PP case probabilistic predictor probabilistic relation	$S\big(p_P, t\big)$	$\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} \big[S\big(h(\mathbf{x}), t\big) \big]$	$\mathop{\mathbb{E}}_{(\mathbf{x}, t) \sim p_J} \big[S\big(h(\mathbf{x}), t\big) \big]$

Table 1: The four combinations of relation model and predictor type. By definition, given a predictor h, $h(\mathbf{x}) = y$ for FF and FP, while $h(\mathbf{x}) = p_P$ for PF and PP. Moving across relation types changes only the per-element risk; moving across predictor types changes the loss from a scalar comparison L to a scoring rule S. The expected risk is always the expectation of the per-element risk with respect to p_M (recall that $p_J(\mathbf{x}, t) = p_M(\mathbf{x}) p_C(t|\mathbf{x})$).

[1] As we shall see, in classification, where $\mathcal{Y}$ is discrete, a loss function comparing pairs of discrete values lacks properties necessary for learning. Thus, functional predictors are defined as predicting real scores — that the decision rule d maps to values in $\mathcal{Y}$ — which are then referred to in the **surrogate** loss function.

2.2 The empirical risk as an estimator of the expected risk

The expected risk depends on L, on p_M and on p_C (or f). L is ours to choose and p_M can usually be assumed known, but f and p_C are unknown by assumption, otherwise the task would be trivial. What we do have is the **empirical risk**: the average loss over the training set, viewed as a sample drawn from p_J,

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t). \tag{16}$$

This is a sensible substitute, since the expected risk factors as

$$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J} \big[L\big(h(\mathbf{x}), t\big)\big] = \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M} \left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h(\mathbf{x}), t\big)\big] \right], \tag{17}$$

and the inner expectation reduces to $L(h(\mathbf{x}), z)$ whenever $p_C(\cdot|\mathbf{x})$ places all its mass on z. Hence the empirical risk is the sample mean used as an estimator of the expectation defining the risk, with the same definition in all four cases (Table 2).

Setting	Loss	Prediction	Empirical risk $\overline{\mathcal{R}}_{\mathcal{T}}(h)$
FF case functional predictor functional relation	$L\big(y,\, t\big)$	$y = h(\mathbf{x})$	$\frac{1}{\lvert\mathcal{T}\rvert} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
FP case functional predictor probabilistic relation	$L\big(y,\, t\big)$	$y = h(\mathbf{x})$	$\frac{1}{\lvert\mathcal{T}\rvert} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
PF case probabilistic predictor functional relation	$S\big(p_P, t\big)$	$p_P = h(\mathbf{x})$	$\frac{1}{\lvert\mathcal{T}\rvert} \sum_{(\mathbf{x},t)\in\mathcal{T}} S\big(p_P, t_i\big)$
PP case probabilistic predictor probabilistic relation	$S\big(p_P, t\big)$	$p_P = h(\mathbf{x})$	$\frac{1}{\lvert\mathcal{T}\rvert} \sum_{(\mathbf{x},t)\in\mathcal{T}} S\big(p_P, t_i\big)$

Table 2: The empirical risk in all four combinations of relation model and predictor type. As can be seen, its structure is the same in all of them: comparing the prediction returned by h with the observed value of the target.

2.2.1 General properties of the sample mean for i.i.d. variables

We first recall general properties of the sample mean of i.i.d. random variables, which apply to the empirical risk as a particular sample mean.

Unbiasedness. Let Z be a random variable with expectation $\mathop{\mathbb{E}}_{Z\sim p}[Z] = \mu$, and let $Z_1, \ldots, Z_n$ be n independent draws from $p(\cdot)$. The average

$$\overline{Z} = \frac{1}{n} \sum_{i=1}^{n} Z_i \tag{18}$$

is an **unbiased estimator** of μ. That is,

$$\mathop{\mathbb{E}}_{(Z_1,\ldots,Z_n)\sim p^n} \big[\overline{Z}\big] = \mu. \tag{19}$$

Thus, the expectation of the sample average equals the true mean μ, which is precisely what it means for an estimator to be unbiased. Note that independence is not required for this result, provided that each Z_i has expectation μ.

The same holds when $Z = g(X)$ for some fixed function g of another random variable X: if $X_1, \ldots, X_n$ are independent draws from the distribution of X, then averaging $g(X_1), \ldots, g(X_n)$ yields an unbiased estimator of $\mathbb{E}_X[g(X)]$, provided that the expectation exists.

Consistency. Unbiasedness says nothing about how close $\overline{Z}$ is to μ for a particular sample. What happens as the sample grows is described by consistency: a sequence of estimators $\hat{\mu}_n$ converges in probability to μ, written $\hat{\mu}_n \xrightarrow{p} \mu$, if for every $\varepsilon > 0$,

$$\lim_{n\to\infty} P\big(|\hat{\mu}_n - \mu| > \varepsilon\big) = 0. \tag{20}$$

By the Weak Law of Large Numbers, the average of n i.i.d. random variables with finite mean converges in probability to that mean. As a consequence,

$$\lim_{n\to\infty} P\big(|\frac{1}{n}\sum_{i=1}^{n} Z_i - \mu| > \varepsilon\big) = 0. \tag{21}$$

Concentration around the mean. Consistency says nothing about how quickly $\overline{Z}$ settles around μ; here the variance enters through the notion of concentration. The elementary tool quantifying this is Chebyshev's inequality.

Chebyshev's inequality and the variance of a sample mean

For *any* random variable X with finite mean μ and finite variance σ^2, and any $\varepsilon > 0$, Chebyshev's inequality states

$$P\big(|X - \mu| > \varepsilon\big) \;\le\; \frac{\sigma^2}{\varepsilon^2}, \tag{22}$$

with no assumption on the shape of the distribution of X.
If $Z_1, \dots, Z_n$ are i.i.d. each with mean μ and variance σ, then

$$\mathrm{Var}(\overline{Z}) \;=\; \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(Z_i) \;=\; \frac{\sigma^2}{n}. \tag{23}$$

Applying Chebyshev's inequality with $X = \overline{Z}$,

$$P\big(|\overline{Z} - \mu| > \varepsilon\big) \;\le\; \frac{\sigma^2}{n\varepsilon^2}, \tag{24}$$

which recovers consistency with an explicit rate: the probability of a sizeable deviation shrinks as $1/n$, and $\overline{Z}$ concentrates around μ on the scale $\sigma/\sqrt{n}$, its standard error.

2.2.2 Specializing to the empirical risk

The empirical risk is an instance of this construction, with g specialized to the loss L and X to the pair $(\mathbf{x}, t)$. Let $(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)$ be the pairs in $\mathcal{T}$, drawn i.i.d. from p_J, and let L be a fixed loss. Given a predictor h, let us consider as usual the expected and the empirical risks

$$\mathcal{R}(h) = \mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\big[L(h(\mathbf{x}), t)\big], \qquad \overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{n}\sum_{i=1}^{n} L(h(\mathbf{x}_i), t_i). \tag{25}$$

The losses $L(h(\mathbf{x}_i), t_i)$ are i.i.d., and $\overline{\mathcal{R}}_{\mathcal{T}}(h)$ is their sample mean, with $\mathcal{R}(h)$ in the role of μ. Three consequences follow.

Unbiasedness. For any loss and any p_J for which $\mathcal{R}(h)$ exists,

$$\mathop{\mathbb{E}}_{\mathcal{T}\sim p_J^n}\big[\overline{\mathcal{R}}_{\mathcal{T}}(h)\big] = \mathcal{R}(h), \tag{26}$$

the expectation being taken over the random draw of the training set, whatever its size.

Consistency. By the Weak Law of Large Numbers, assuming only that $\mathcal{R}(h)$ exists,

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) \xrightarrow{p} \mathcal{R}(h), \qquad \text{as } |\mathcal{T}| \to \infty. \tag{27}$$

Rate. Writing $\sigma_L^2(h) = \underset{(\mathbf{x},t)\sim p_J}{Var}\left[L(h(\mathbf{x}),t)\right]$ for the variance of the per-example loss, we have $\underset{\mathcal{T}\sim p_J^n}{Var}\left[\overline{\mathcal{R}}_{\mathcal{T}}(h)\right] = \frac{\sigma_L^2(h)}{n}$ and hence

$$P\left(\left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\right| > \varepsilon\right) \;\leq\; \frac{\sigma_L^2(h)}{n\varepsilon^2}\,; \tag{28}$$

the empirical risk of a *fixed* h concentrates around its expected risk on the scale $\frac{\sigma_L(h)}{\sqrt{n}}$.

Our ultimate goal, however, is performance on unseen data, expressed by the expected risk: the relation between the two is the main topic of **statistical learning theory**.

2.3 Minimization of the empirical risk and its issues

The empirical risk allows us to identify the "best" predictor in a given set $\mathcal{H}$ (the **hypothesis class**) via **empirical risk minimization** (ERM),

$$h_{\mathcal{T}} \;=\; \underset{h\in\mathcal{H}}{\operatorname{argmin}}\, \overline{\mathcal{R}}_{\mathcal{T}}(h). \tag{29}$$

Two separate questions arise. The first is *statistical*: the predictor we actually want is $h^*_{\mathcal{H}} = \underset{h\in\mathcal{H}}{\operatorname{argmin}}\, \mathcal{R}(h)$, the one with minimal true risk, and we ask whether $h_{\mathcal{T}}$ approaches it as the training set grows. The second is *computational*: $\mathcal{H}$ may be a space of functions in which optimization is hard. We take up the statistical question first, since its answer already tells us which hypothesis classes are admissible.

2.3.1 Reaching the target from a sample

The predictor we would like to find is $h^*_{\mathcal{H}}$, the minimizer of the true risk over $\mathcal{H}$, while ERM returns $h_{\mathcal{T}}$, the minimizer of the empirical risk. Equation (28) ensures that the empirical risk concentrates around the true risk, but only for a predictor h fixed before $\mathcal{T}$ is drawn:

$$P\left(\left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\right| > \varepsilon\right) \xrightarrow{n\to\infty} 0 \qquad \text{for every fixed } h \text{ and } \varepsilon > 0. \tag{30}$$

This guarantee does not cover $h_{\mathcal{T}}$, which is selected by looking at $\mathcal{T}$: a different training set returns, in general, a different predictor. Informally, deriving $h_{\mathcal{T}} \to h^*_{\mathcal{H}}$ from Equation (30) would amount to exchanging a limit and a minimization,

$$\lim_{n\to\infty}\ \underset{h\in\mathcal{H}}{\operatorname{argmin}}\, \overline{\mathcal{R}}_{\mathcal{T}}(h) \;\overset{?}{=}\; \underset{h\in\mathcal{H}}{\operatorname{argmin}}\ \lim_{n\to\infty}\, \overline{\mathcal{R}}_{\mathcal{T}}(h), \tag{31}$$

where the right-hand side is $h^*_{\mathcal{H}}$ and the left-hand side is what ERM returns on larger and larger samples. The two operations do not commute in general. Figure 6 shows why: the empirical risk is a noisy version of the true risk, and its minimum may lie where the noise happens to be most favourable rather than where the true risk is smallest.

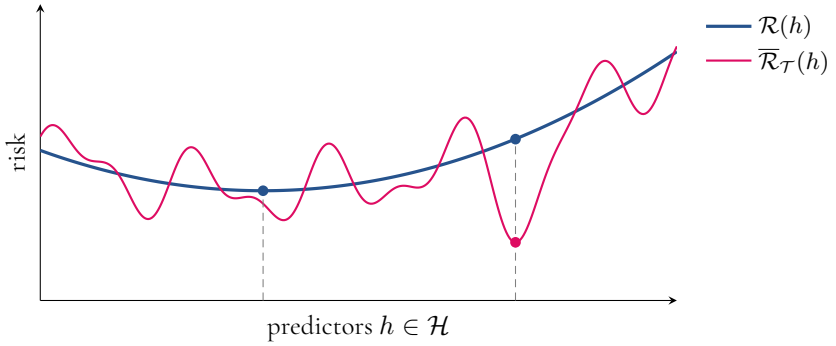


Figure 6: True risk (smooth) and empirical risk (noisy) over a hypothesis class. ERM selects $h_{\mathcal{T}}$, where the empirical risk is lowest: its empirical risk underestimates its true risk, which is larger than that of $h^*_{\mathcal{H}}$.

A sufficient condition for ERM to work follows from comparing the excess risk of $h_{\mathcal{T}}$ with the gaps between empirical and true risk.

Excess risk and uniform deviation

For any training set $\mathcal{T}$, add and subtract the empirical risks of $h_{\mathcal{T}}$ and $h^*_{\mathcal{H}}$:

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) = \underbrace{\left[\mathcal{R}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\right]}_{\text{(a)}} + \underbrace{\left[\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h^*_{\mathcal{H}})\right]}_{\text{(b)}} + \underbrace{\left[\overline{\mathcal{R}}_{\mathcal{T}}(h^*_{\mathcal{H}}) - \mathcal{R}(h^*_{\mathcal{H}})\right]}_{\text{(c)}}.$$

Term (b) is never positive, since $h_{\mathcal{T}}$ minimizes the empirical risk over $\mathcal{H} \ni h^*_{\mathcal{H}}$, and terms (a) and (c) are bounded in absolute value by $\sup_{h\in\mathcal{H}} |\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)|$. Therefore, for every $\mathcal{T}$,

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) \;\le\; 2 \sup_{h\in\mathcal{H}} \left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\right|, \tag{32}$$

and, considering the probability over the draw of $\mathcal{T}$ with $|\mathcal{T}| = n$,

$$P\Big(\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}}) > 2\varepsilon\Big) \;\le\; P\Big(\sup_{h\in\mathcal{H}} \left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h)\right| > \varepsilon\Big). \tag{33}$$

If the right-hand side of Equation (33) tends to 0 for every $\varepsilon > 0$, that is, if the empirical risk converges to the true risk **uniformly** over $\mathcal{H}$, the true risk of $h_{\mathcal{T}}$ converges in probability to that of $h^*_{\mathcal{H}}$. The idea is simple: if no predictor in $\mathcal{H}$ has an empirical risk far from its true risk, then the predictor selected by ERM, whichever it is, has a reliable empirical risk too. Whether uniform convergence holds depends on $\mathcal{H}$.

When $\mathcal{H}$ is unrestricted, the gap between empirical and true risk does not shrink at all. Consider binary classification with $t \in \{0, 1\}$ and the 0–1 loss, let $\mathcal{H}$ contain all functions $\mathcal{X} \to \{0, 1\}$, and let $\mathbf{x}$ have a continuous distribution. Assume the extreme case of noisy labels, $\pi(\mathbf{x}) = p(t = 1 \mid \mathbf{x}) = \frac{1}{2}$ for all $\mathbf{x}$: the label is a fair coin toss, independent of the input. Nothing can then be learned. Any classifier errs with probability $\frac{1}{2}$ on each input, so every $h \in \mathcal{H}$, the Bayes classifier (Section 2.4) included, has true risk $\frac{1}{2}$.

For any training set, however, $\mathcal{H}$ also contains **lookup tables**, which memorize the training set. Consider the simplest one, h_c, which returns t_i on each $\mathbf{x}_i$ and a fixed constant $c \in \{0, 1\}$ everywhere else.

On the training set h_c never errs, since it returns exactly the observed labels: its empirical risk is 0. As the 0–1 risk cannot be negative, this is the smallest possible value, and h_c is an ERM solution. The constant classifier $h(\mathbf{x}) = c$, which does not memorize the data, instead errs on about half of the training points, since each label is a coin toss: its empirical risk is close to its true risk $\frac{1}{2}$. ERM looks only at the empirical risk, and therefore strictly prefers the lookup table: fitting the noise is rewarded.

On new data, however, the two classifiers coincide. Since $\mathbf{x}$ is continuous, a new input coincides with one of the n training points with probability zero. Almost surely, then, h_c predicts c, whatever the input: the memorized labels play no role in prediction, and the true risk of h_c is $\frac{1}{2}$, as for every classifier. ERM thus returns a predictor that looks perfect on the data but does no better than tossing a coin. Nor can more data fix the problem. For every n, h_c has empirical risk 0 and true risk $\frac{1}{2}$, so the largest gap over $\mathcal{H}$ never drops below $\frac{1}{2}$, whatever the sample size: a larger sample only provides a new lookup table with the same property.

Lookup tables in detail

Fix a training set $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$ and $c \in \{0, 1\}$, and define

$$h_c(\mathbf{x}) = \begin{cases} t_i & \text{if } \mathbf{x} = \mathbf{x}_i, \\ c & \text{otherwise,} \end{cases}$$

a function $\mathcal{X} \to \{0, 1\}$, hence in $\mathcal{H}$.

True risk. Since $p(t \neq h(\mathbf{x}) \mid \mathbf{x}) = \frac{1}{2}$ whatever $h(\mathbf{x})$ is, every $h \in \mathcal{H}$ has $\mathcal{R}(h) = \frac{1}{2}$. In particular, a new $\mathbf{x}$ coincides with a training point with probability zero, so almost surely $h_c(\mathbf{x}) = c$, and $\mathcal{R}(h_c) = p(t \neq c) = \frac{1}{2}$.

Empirical risk. On each training point h_c returns exactly t_i, so $\overline{\mathcal{R}}_{\mathcal{T}}(h_c) = 0$, the minimum of the 0–1 risk. For the constant classifier $h(\mathbf{x}) = c$, instead, $\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{n}\sum_i \mathbf{1}[t_i \neq c]$ is the average of

n independent Bernoulli variables with parameter $\frac{1}{2}$: its mean is $\frac{1}{2}$ and its standard deviation $\frac{1}{2\sqrt{n}}$. With $n = 100$, it lies between 0.4 and 0.6 with probability about 95%.

Consequences. Unlike the quadratic loss, the 0–1 loss is bounded, and so is the gap; it does not decrease, however, since for every $\mathcal{T}$ and every n

$$\sup_{h \in \mathcal{H}} \left| \overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}(h) \right| \;\geq\; \left| \overline{\mathcal{R}}_{\mathcal{T}}(h_c) - \mathcal{R}(h_c) \right| \;=\; \tfrac{1}{2},$$

and the right-hand side of Equation (33) never tends to 0.

Restricting $\mathcal{H}$ is therefore necessary for ERM to work. The number of elements is not the right measure of richness, since many infinite classes, such as linear classifiers, converge uniformly: what matters is a suitable complexity measure, such as the VC dimension. This is the first, statistical reason for restricting $\mathcal{H}$; the next subsection gives a second, computational one.

2.3.2 Restricting $\mathcal{H}$ for computational reasons too

A second, independent reason points in the same direction: searching a space of functions for the empirical risk minimizer is not directly amenable to standard optimization tools (gradients, line searches), which are built for spaces of numbers. The standard remedy is to restrict $\mathcal{H}$ to a **parametric family**: fix a functional form $h_{\boldsymbol{\theta}}$ and let $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d)$ range over $\mathbb{R}^{d+1}$. Each $\boldsymbol{\theta}$ picks out one function in $\mathcal{H}$, the search over functions becomes a search over parameter vectors, and training becomes finding the $\boldsymbol{\theta}$ minimizing the empirical risk — whether $h_{\boldsymbol{\theta}}$ is a linear model, a fixed-depth tree, or a neural network. This restriction is also exactly the kind of complexity control required above: a family indexed by $d + 1$ parameters typically has complexity growing only with d, so a modest d makes both gradient-based search and uniform convergence possible.

For the parametric case that occupies most of these notes, we reformulate empirical risk minimization as minimizing the overall loss

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}), \tag{34}$$

where we sum the individual loss terms $L_i = L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$ over each data point in the training set, making explicit the dependence on both the parameters and the training example.

2.3.3 Comparing predictors on an independent test set

Every failure traced above has one root cause: the empirical risk was asked to serve *two* roles at once — selecting $h_{\mathcal{T}}$ and certifying its quality — on the same sample. The machinery developed so far (union bound, optimism of the minimum, uniform convergence) repairs this *theoretically*. There is a far cheaper, purely empirical fix: evaluating $h_{\mathcal{T}}$ on a second, independent **test set** $\mathcal{T}'$, drawn i.i.d. from p_J and never consulted during selection. Conditionally on $\mathcal{T}$, $h_{\mathcal{T}}$ is simply *some fixed function* with respect to $\mathcal{T}'$, and the hypothesis of Equation (30) is restored by construction:

$$P\left(\left| \overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}}) - \mathcal{R}(h_{\mathcal{T}}) \right| > \varepsilon \;\middle|\; \mathcal{T} \right) \xrightarrow{|\mathcal{T}'| \to \infty} 0, \tag{35}$$

with no supremum over $\mathcal{H}$ and no complexity measure involved. Consequently $\mathcal{H}$ can be as rich as convenient during training — even the interpolating construction above — since $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}})$ reports the truth regardless. The gap $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})$ is the practical substitute for the uniform-convergence quantity of Equation (33): generalization is measured after the fact rather than proved in advance.

The fix, however, holds only for a *single*, already-fixed $h_{\mathcal{T}}$ evaluated *once*. If $\mathcal{T}'$ is used to *choose* among K candidates (different algorithms, architectures, or hyperparameters), this is ERM over those candidates with $\mathcal{T}'$ in the role of $\mathcal{T}$, and every result above transfers verbatim: comparing a small K, fixed in advance, once, is the safe regime, whereas repeatedly inspecting performance on $\mathcal{T}'$ and adjusting the model reproduces the optimism of ERM described above. A test set used for model selection has become a second *training* set. Hence the standard three-way split: a *training* set, on which $\mathcal{H}$ can be as rich as convenient; a *validation* set for model and hyperparameter selection, safe only as long as the number of compared configurations stays small; and a *test* set used exactly once, on the final model — the one point where Equation (30) applies without qualification, giving an unbiased estimate of the true risk.

2.4 Bayes predictors and Bayes risk

The expected risk cannot be minimized in practice, since it requires knowing p_M and p_C. It is nonetheless worth asking what the best possible predictor would look like if this knowledge *were* available: although not computable, it provides an ideal benchmark for learned predictors.

Formally, the **Bayes predictor** h^B minimizes the expected risk over the class of *all* measurable functions[2] $h : \mathcal{X} \to \mathbb{R}$:

$$h^B = \operatorname*{argmin}_h \mathcal{R}(h). \tag{36}$$

The minimal value $\mathcal{R}^B = \mathcal{R}(h^B)$ is the **Bayes risk**: the error left even for the best conceivable predictor, arising whenever $p_C(\cdot|\mathbf{x})$ spreads mass over more than one value of t. If two patients with identical features sometimes recover and sometimes do not, no predictor can foresee the individual outcome, only report the correct probability of recovery. The Bayes risk is *irreducible*, reflecting noise intrinsic to the phenomenon rather than ignorance that more data could dispel.

Two remarks are in order. First, h^B depends on the loss L, not only on p_J: different losses extract different statistics from the same posterior — squared loss yields the conditional mean, absolute loss the conditional median, and so on.

Example

Suppose, for fixed $\mathbf{x}$, that $t = 0$ with probability 0.9 and $t = 10$ with probability 0.1. Under squared loss the Bayes-optimal prediction is the conditional mean, $0.9 \cdot 0 + 0.1 \cdot 10 = 1$, a value t never takes; under absolute loss it is the conditional median, 0. Same posterior, different losses, different optimal predictions.

Second, computing h^B requires $p_C(\cdot|\mathbf{x})$, exactly what learning assumes to be unavailable: the Bayes predictor is a benchmark, never a practical output of training.

Restricting the search to $\mathcal{H}$ generally worsens the achievable risk, since $\mathcal{H}$ may not contain h^B: no linear function reproduces a genuinely nonlinear relation. Let $h^*_{\mathcal{H}} = \operatorname*{argmin}_{h \in \mathcal{H}} \mathcal{R}(h)$ be the best predictor in $\mathcal{H}$, still computed under perfect knowledge of p_J, and $h_{\mathcal{T}} \in \mathcal{H}$ the predictor actually returned by learning on the finite sample $\mathcal{T}$: the only one of the three we can compute, and generally the worst. Inserting $h^*_{\mathcal{H}}$ as an intermediate reference point decomposes the excess risk of $h_{\mathcal{T}}$ over the Bayes risk:

$$\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}^B = \underbrace{\left[\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}})\right]}_{\text{estimation error}} + \underbrace{\left[\mathcal{R}(h^*_{\mathcal{H}}) - \mathcal{R}^B\right]}_{\text{approximation error}}. \tag{37}$$

Both terms are nonnegative, since $h^*_{\mathcal{H}}$ and h^B are risk minimizers over their respective search spaces.

The **approximation error**, $\mathcal{R}(h^*_{\mathcal{H}}) - \mathcal{R}^B$, measures the risk given up by searching within $\mathcal{H}$ rather than over all functions. It compares two idealized predictors, so it depends only on how well $\mathcal{H}$ matches the true relation between $\mathbf{x}$ and t, and decreases as $\mathcal{H}$ becomes richer. It is associated with **underfitting**: a restrictive class fails to represent the true relation however carefully it is fit. The **estimation error**, $\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}(h^*_{\mathcal{H}})$, compares two predictors in the same $\mathcal{H}$ differing only in the information used to select them: exact knowledge of p_J versus a finite, noisy sample. It decreases with more data but increases with model flexibility, since richer classes are harder to estimate reliably; it is the term associated with **overfitting**.

With a smooth quadratic relation plus noise, for instance, linear functions have large approximation error but, having few parameters, small estimation error; degree-ten polynomials have negligible approximation error but, with few examples, tend to wildly oscillating fits, i.e. large estimation error. Enlarging $\mathcal{H}$ shrinks the first term while inflaming the second: a tension closely related, though not identical, to the **bias-variance tradeoff** of Section 3. What remains once both controllable errors vanish is the Bayes risk, which no choice of $\mathcal{H}$, amount of data, or algorithm can remove. Recognizing which of the two controllable terms dominates — underfitting or overfitting — is a central diagnostic question of supervised learning.

[2] A function $h : \mathcal{X} \to \mathbb{R}$ is **measurable** if, for every interval $(a, b) \subset \mathbb{R}$, the set $\{\mathbf{x} \in \mathcal{X} : h(\mathbf{x}) \in (a, b)\}$ belongs to the collection of subsets over which probabilities are defined. This is a very mild regularity condition, imposing no constraint on the shape or complexity of h: every function one would realistically compute is measurable, so "all measurable functions" essentially means "all functions".

3 The bias-variance decomposition

The decomposition of Equation (37) concerns the predictor $h_{\mathcal{T}}$ learned from one specific training set. A complementary view analyses the learning algorithm itself: since $h_{\mathcal{T}}$ is random, its error can be studied *on average over training sets*, pointwise at each $\mathbf{x}$, separating what the procedure systematically predicts (**bias**) from its sensitivity to the particular sample (**variance**).

3.1 The general decomposition

Fix an item $\mathbf{x}$ and consider the predictions $h_{\mathcal{T}}(\mathbf{x})$ as $\mathcal{T}$ varies over training sets of size n. Their "center" is summarized by the **main prediction**

$$\bar{h}(\mathbf{x}) \;=\; \operatorname*{argmin}_{z} \; \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \big[L\big(h_{\mathcal{T}}(\mathbf{x}),\, z\big) \big] \,, \tag{38}$$

the value which, taken as a target, is on average closest to the predictions under L.

Functional relation. If $t = f(\mathbf{x})$, then $h^B(\mathbf{x}) = f(\mathbf{x})$ for any loss with $L(z,z) = 0 < L(y,z)$ when $y \neq z$, and adding and subtracting $L(\bar{h}(\mathbf{x}), f(\mathbf{x}))$ gives

$$\mathop{\mathbb{E}}_{\mathcal{T}} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) \big] \;=\; \underbrace{L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big)}_{\text{bias}} \;+\; \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) - L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big) \big]}_{\text{variance}} \,. \tag{39}$$

Probabilistic relation: including the irreducible risk. If $t \sim p_C(\cdot|\mathbf{x})$, even h^B incurs a nonzero risk. Adding and subtracting the loss of both $h^B(\mathbf{x})$ and $\bar{h}(\mathbf{x})$ inside the expectation over t yields

$$\mathop{\mathbb{E}}_{\mathcal{T}} \Big[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) \big] \Big] = \mathop{\mathbb{E}}_{\mathcal{T}} \big[\mathcal{R}_{p_C}(\mathbf{x}; h_{\mathcal{T}}) \big] = \underbrace{\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h^B(\mathbf{x}), t\big) \big]}_{\text{irreducible risk } C(p_C(\cdot|\mathbf{x}))} + \underbrace{\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(\bar{h}(\mathbf{x}), t\big) - L\big(h^B(\mathbf{x}), t\big) \big]}_{\text{bias } B(\mathbf{x})}$$
$$+ \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}} \Big[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})} \big[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) - L\big(\bar{h}(\mathbf{x}), t\big) \big] \Big]}_{\text{variance } V(\mathbf{x})} \,.$$

$C(p_C(\cdot|\mathbf{x}))$ is the pointwise Bayes risk of Section 2.4, fixed by p_C and L alone; Equation (39) is recovered when $p_C(\cdot|\mathbf{x})$ is a point mass at $f(\mathbf{x})$, which makes $C(\mathbf{x}) = 0$.

The identity holds for *every* loss and every choice of $\bar{h}$, since the terms added and subtracted cancel out: by itself, it says nothing about the problem. Its content lies entirely in what the three terms turn out to be.

3.2 Interpretability

For the names "bias" and "variance" to be justified, the terms of the decomposition should satisfy the following properties:

(i) $B(\mathbf{x}) \geq 0$, with $B(\mathbf{x}) = 0$ if and only if $\bar{h}(\mathbf{x}) = h^B(\mathbf{x})$: the bias depends only on where the main prediction sits;

(ii) $V(\mathbf{x}) \geq 0$, and $V(\mathbf{x})$ depends only on the spread of the predictions $h_{\mathcal{T}}(\mathbf{x})$ around $\bar{h}(\mathbf{x})$, not on $p_C(\cdot|\mathbf{x})$;

(iii) the terms combine additively, so that reducing one, with the others fixed, reduces the total.

Property (i) holds for any loss, by definition of h^B. Properties (ii) and (iii) do not: for a general loss the variance term mixes the fluctuation of the predictions with the noise of the target, and can even be negative. There is, however, a broad class of losses for which all three properties hold.

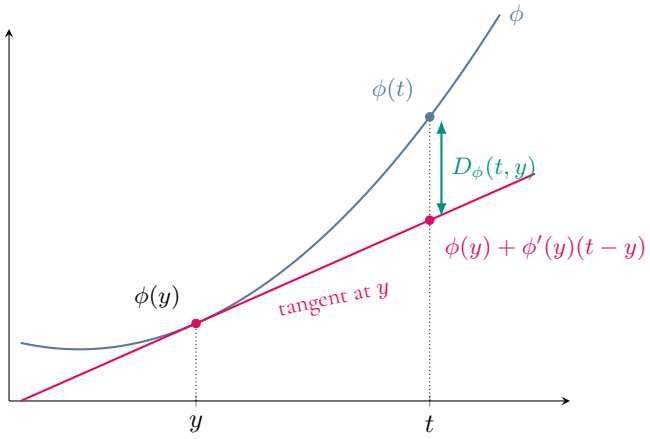


Figure 7: Bregman divergence: the gap at t between the convex function ϕ and its tangent line at y. By convexity the tangent never lies above ϕ, so the gap is non-negative, and it vanishes only when $y = t$.

Bregman divergences. Informally, a Bregman divergence measures the cost of predicting y when the truth is t by taking a strictly convex function ϕ and asking how badly its tangent line at y approximates ϕ at t:

$$D_\phi(t, y) \;=\; \phi(t) - \phi(y) - \phi'(y)\,(t - y) \;\geq\; 0. \tag{40}$$

By convexity this gap is never negative, and it vanishes only when $y = t$. Different choices of ϕ give different losses: $\phi(y) = y^2$ gives the quadratic loss $(t - y)^2$.

What makes these losses special is that their expected loss always splits into a part due to the noise and a part depending only on how far the prediction is from the conditional mean $\mu(\mathbf{x})$:

$$\mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} [D_\phi(t, y)] \;=\; \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} [D_\phi(t, \mu(\mathbf{x}))] \;+\; D_\phi\big(\mu(\mathbf{x}), y\big). \tag{41}$$

The cross term cancels because $\mathbb{E}_t\,[t - \mu(\mathbf{x})] = 0$. Two consequences follow: the Bayes predictor is the conditional mean $h^B(\mathbf{x}) = \mu(\mathbf{x})$ whatever ϕ is, and the same split applies to the predictions themselves, yielding a decomposition into irreducible risk, bias, and variance in which all three terms are nonnegative divergences, so that properties (i)–(iii) all hold.

Losses with an interpretable decomposition. Several losses widely used in practice are Bregman divergences. Of specific interest here are the quadratic loss, $\phi(y) = y^2$, for which $\bar{h}(\mathbf{x})$ is the ordinary mean of the predictions and the decomposition is the classical noise + bias^2 + variance, and the log loss (cross-entropy) for probabilistic predictors, generated by the negative entropy $\phi(p) = \sum_k p_k \log p_k$, for which D_ϕ is the Kullback–Leibler divergence. Many other common losses — the absolute loss, the Huber loss, the hinge loss, the 0–1 loss — are not Bregman divergences: the decomposition still holds as an identity for them, but its variance term may depend on the noise of the target and may even be negative.

Relation with the approximation/estimation decomposition. The two decompositions look at the same error from different angles. Approximation and estimation error refer to the predictor learned from one training set and compare it with $h^*_{\mathcal{H}}$ and h^B; bias and variance describe the learning procedure on average over training sets, using $\bar{h}$ as reference. Averaging the bias-variance decomposition over $\mathbf{x} \sim p_M$ (the irreducible term becomes $\mathcal{R}^B_{p_J}$) and the error decomposition of Equation (37) over $\mathcal{T}$ gives two expressions of the same quantity, the expected excess risk:

$$\text{approximation error} + \mathop{\mathbb{E}}_{\mathcal{T}} [\text{estimation error}] \;=\; \mathop{\mathbb{E}}_{\mathbf{x}} [B(\mathbf{x})] + \mathop{\mathbb{E}}_{\mathbf{x}} [V(\mathbf{x})]\,. \tag{42}$$

The totals always coincide; the individual terms, in general, do not. The bias compares $\bar{h}$ with h^B, the approximation error compares $h^*_{\mathcal{H}}$ with h^B, and $\bar{h}$ need neither coincide with $h^*_{\mathcal{H}}$ nor even belong to $\mathcal{H}$ (an average of neural networks is not, in general, a neural network of the same architecture).